Java Datentypen

Autor

Luis Staudt

19. Dezember 2025

Inhaltsverzeichnis

Inhaltsverzeichnis 2
Java Datentypen 3
Primitive Datentypen 3
int - Ganzzahlen 4
double - Fließkommazahlen 6
boolean - Wahrheitswerte 8
char - Einzelnes Zeichen 10
byte - Sehr kleine Ganzzahlen 12
short - Kleine Ganzzahlen 13
long - Sehr große Ganzzahlen 14
float - Einfache Fließkommazahlen 15
String - Zeichenketten 16
Typkonvertierung 19
Übersicht: Primitive Datentypen 21
Häufige Fehler 21

Java Datentypen

Primitive Datentypen

Java unterscheidet zwischen primitiven Datentypen und Referenztypen (Objekten). Primitive Datentypen sind die grundlegenden Bausteine und speichern direkt Werte.

int - Ganzzahlen

Was ist int?

Der Datentyp `int` (Integer) speichert ganze Zahlen ohne Nachkommastellen.

Wertebereich:

- Von: -2.147.483.648
- Bis: 2.147.483.647
- Speichergröße: 32 Bit (4 Bytes)

Syntax:

```
int variablenName = wert;
```

Beispiele:

```
// Deklaration und Initialisierung
int alter = 25;
int temperatur = -5;
int punkte = 0;

// Ohne Initialisierung
int zahl;
zahl = 42;

// Berechnungen
int summe = 10 + 20;          // summe = 30
int differenz = 50 - 15;      // differenz = 35
int produkt = 6 * 7;          // produkt = 42
int quotient = 20 / 4;        // quotient = 5
int rest = 17 % 5;            // rest = 2

// Mit Variablen rechnen
int a = 10;
int b = 3;
int ergebnis = a + b;         // ergebnis = 13

// Inkrement und Dekrement
int zaehler = 5;
zaehler++;                    // zaehler = 6
zaehler--;                    // zaehler = 5

```

```
// Zusammengesetzte Operatoren
int wert = 10;
wert += 5;                    // wert = 15 (entspricht: wert = wert + 5)
wert -= 3;                    // wert = 12
wert *= 2;                    // wert = 24
wert /= 4;                    // wert = 6
```

Worauf achten:

- Division von int durch int ergibt int: `5 / 2 = 2` (nicht 2.5!)
- Bei Überschreitung des Wertebereichs: **Overflow** (Wert springt um)
- Standard-Wert bei Arrays: `0`
- Keine Nachkommastellen möglich

double - Fließkommazahlen

Was ist double?

Der Datentyp `double` speichert Zahlen mit Nachkommastellen (Gleitkommazahlen mit doppelter Genauigkeit).

Wertebereich:

- Sehr große und sehr kleine Zahlen mit ca. 15 Dezimalstellen Genauigkeit
- Von: $\pm 4.9 \times 10^{-324}$
- Bis: $\pm 1.7 \times 10^{308}$
- Speichergröße: 64 Bit (8 Bytes)

Syntax:

```
double variablenName = wert;
```

Beispiele:

```
// Deklaration und Initialisierung
double pi = 3.14159;
double temperatur = -5.5;
double gewicht = 75.8;
double ergebnis = 0.0;

// Berechnungen
double summe = 10.5 + 20.3;        // summe = 30.8
double produkt = 2.5 * 4.0;        // produkt = 10.0
double quotient = 7.0 / 2.0;       // quotient = 3.5

// Wichtig: Division mit Nachkommastellen
double a = 5.0;
double b = 2.0;
double teil = a / b;               // teil = 2.5

// int zu double konvertieren
int ganzzahl = 5;
double kommazahl = ganzzahl / 2.0; // kommazahl = 2.5
// ODER
double kommazahl2 = (double) ganzzahl / 2; // kommazahl2 = 2.5

// Wissenschaftliche Notation
```

```
24 double klein = 1.5e-4;              // = 0.00015
25 double gross = 3.2e8;               // = 320000000.0
26
27 // Mathematische Funktionen
28 double wurzel = Math.sqrt(16.0);    // wurzel = 4.0
29 double potenz = Math.pow(2, 3);     // potenz = 8.0
30 double absolut = Math.abs(-5.5);    // absolut = 5.5
```

Worauf achten:

- Punkt als Dezimaltrennzeichen, **nicht** Komma: `3.14` nicht `3,14`
- Rundungsfehler möglich: `0.1 + 0.2 = 0.30000000000000004`
- Division `5 / 2` ergibt `2`, aber `5.0 / 2.0` ergibt `2.5`
- Standard-Wert bei Arrays: `0.0`
- Für Geldbeträge besser `BigDecimal` verwenden

boolean - Wahrheitswerte

Was ist boolean?

Der Datentyp `boolean` speichert Wahrheitswerte: entweder `true` (wahr) oder `false` (falsch).

Wertebereich:

- Nur zwei mögliche Werte: `true` oder `false`
- Speichergröße: 1 Bit (theoretisch), praktisch abhängig von JVM

Syntax:

```
boolean variablenName = true/false;
```

Beispiele:

```
// Deklaration und Initialisierung
boolean istWahr = true;
boolean istFalsch = false;
boolean hatBestanden = true;

// Vergleichsoperatoren ergeben boolean
int alter = 18;
boolean istErwachsen = (alter >= 18);      // true
boolean istKind = (alter < 12);            // false
boolean istGenau18 = (alter == 18);        // true
boolean istNicht18 = (alter != 18);        // false

// Logische Operatoren
boolean a = true;
boolean b = false;

boolean und = a && b;                      // false (beide muessen true
    sein)
boolean oder = a || b;                     // true (mindestens einer true)
boolean nicht = !a;                        // false (Negation)

// In Bedingungen
int punkte = 85;
boolean bestanden = punkte >= 50;
if (bestanden) {
    System.out.println("Bestanden!");
}
```

```java

// Kombinierte Bedingungen
int temperatur = 25;
boolean angenehm = temperatur >= 18 && temperatur <= 28;

// Boolean in Schleifen
boolean weitermachen = true;
while (weitermachen) {
    // Code
    weitermachen = false;  // Schleife beenden
}

// Vergleich mit Strings
String name = "Max";
boolean istMax = name.equals("Max");       // true
```

Worauf achten:

- Nur `true` oder `false`, **keine** Zahlen (nicht wie in C: 0 = false, 1 = true)
- Kleinschreibung: `true`/`false`, nicht `True`/`False`
- Standard-Wert bei Arrays: `false`
- `==` für primitive Typen, `.equals()` für Objekte (z.B. Strings)

char - Einzelnes Zeichen

Was ist char?

Der Datentyp `char` speichert ein einzelnes Zeichen (Character) im Unicode-Format.

Wertebereich:

- Alle Unicode-Zeichen (Buchstaben, Ziffern, Sonderzeichen)
- Von: 0 bis 65.535 (als Zahlenwert)
- Speichergröße: 16 Bit (2 Bytes)

Syntax:

```
char variablenName = 'Zeichen';  // EINFACHE Anfuehrungszeichen!
```

Beispiele:

```
// Deklaration und Initialisierung
char buchstabe = 'A';
char ziffer = '5';
char sonderzeichen = '!';
char leerzeichen = ' ';

// Mit Unicode-Werten
char euro = '\u20AC';                    //    -Symbol
char herzchen = '\u2764';                //

// Escape-Sequenzen
char neuzeile = '\n';                    // Zeilenumbruch
char tab = '\t';                         // Tabulator
char backslash = '\\';                   // \
char anfuehrung = '\'';                  // '

// Zeichen vergleichen
char note = 'A';
if (note == 'A') {
    System.out.println("Sehr gut!");
}

// Zeichen in Berechnungen (ASCII/Unicode-Werte)
char a = 'A';                            // Unicode-Wert: 65
char b = 'B';                            // Unicode-Wert: 66
int differenz = b - a;                   // differenz = 1
```

```java

// Kleinbuchstabe zu Gro buchstabe
char klein = 'a';                          // Unicode-Wert: 97
char gross = (char) (klein - 32);          // gross = 'A' (Wert: 65)

// Pr fen ob Buchstabe
char zeichen = 'M';
boolean istBuchstabe = (zeichen >= 'A' && zeichen <= 'Z') ||
                       (zeichen >= 'a' && zeichen <= 'z');

// Pr fen ob Ziffer
char z = '7';
boolean istZiffer = (z >= '0' && z <= '9');

// char aus String extrahieren
String text = "Hallo";
char erstes = text.charAt(0);              // erstes = 'H'
```

Worauf achten:

- **Einfache** Anführungszeichen: `'A'`, nicht Ä“
- Nur **ein** Zeichen: `'A'` ist char, Ä“ ist String
- `char` ist **nicht** dasselbe wie `String`
- Standard-Wert bei Arrays: `'\u0000'` (Null-Zeichen)
- Kann als Zahlenwert verwendet werden (Unicode-Wert)

byte - Sehr kleine Ganzzahlen

Was ist byte?

Der Datentyp `byte` speichert sehr kleine ganze Zahlen. Wird verwendet, wenn Speicherplatz wichtig ist.

Wertebereich:

- Von: -128
- Bis: 127
- Speichergröße: 8 Bit (1 Byte)

Syntax:

```
byte variablenName = wert;
```

Beispiele:

```
// Deklaration und Initialisierung
byte alter = 25;
byte temperatur = -15;
byte prozent = 100;

// H ufig bei Datei-Operationen
byte[] daten = new byte[1024];

// Konvertierung von int zu byte
int gross = 100;
byte klein = (byte) gross;

// Vorsicht bei  berschreitung
byte max = 127;
byte ueberlauf = (byte) (max + 1);         // ueberlauf = -128 (Overflow!)
```

Worauf achten:

- Sehr kleiner Wertebereich: nur -128 bis 127
- Bei Überschreitung: Overflow
- Wird oft für Binärdaten verwendet
- Standard-Wert bei Arrays: `0`

short - Kleine Ganzzahlen

Was ist short?

Der Datentyp `short` speichert kleine ganze Zahlen. Ein Mittelweg zwischen `byte` und `int`.

Wertebereich:

- Von: -32.768
- Bis: 32.767
- Speichergröße: 16 Bit (2 Bytes)

Syntax:

```
short variablenName = wert;
```

Beispiele:

```
// Deklaration und Initialisierung
short jahr = 2024;
short distanz = 5000;

// Konvertierung
int gross = 1000;
short klein = (short) gross;
```

Worauf achten:

- Wird selten verwendet, meist wird direkt `int` genommen
- Wertebereich: -32.768 bis 32.767
- Standard-Wert bei Arrays: `0`

long - Sehr große Ganzzahlen

Was ist long?

Der Datentyp `long` speichert sehr große ganze Zahlen, die über den `int`-Bereich hinausgehen.

Wertebereich:

- Von: -9.223.372.036.854.775.808
- Bis: 9.223.372.036.854.775.807
- Speichergröße: 64 Bit (8 Bytes)

Syntax:

```
long variablenName = wert;
long grosseZahl = wertL;  // L am Ende fuer long-Literal
```

Beispiele:

```
// Deklaration und Initialisierung
long weltbevoelkerung = 8000000000L;       // L am Ende!
long millisekunden = System.currentTimeMillis();

// Ohne L wird es als int interpretiert (Fehler bei gro en Zahlen)
// long falsch = 8000000000;               // FEHLER!
long richtig = 8000000000L;                // Korrekt

// Berechnungen
long summe = 1000000000L + 2000000000L;
```

Worauf achten:

- Bei großen Zahlen: `L` am Ende anhängen: `1000000000L`
- Ohne `L` wird Zahl als `int` interpretiert (kann zu Fehlern führen)
- Standard-Wert bei Arrays: `0L`
- Verwenden, wenn Zahlen größer als ca. 2 Milliarden sind

float - Einfache Fließkommazahlen

Was ist float?

Der Datentyp `float` speichert Fließkommazahlen mit einfacher Genauigkeit. Weniger genau als `double`.

Wertebereich:

- Ca. 6-7 Dezimalstellen Genauigkeit
- Von: $\pm 1.4 \times 10^{-45}$
- Bis: $\pm 3.4 \times 10^{38}$
- Speichergröße: 32 Bit (4 Bytes)

Syntax:

```
float variablenName = wertF;  // F am Ende fuer float-Literal
```

Beispiele:

```
// Deklaration und Initialisierung
float pi = 3.14F;                          // F am Ende!
float temperatur = 36.5F;
float gewicht = 75.8F;

// Ohne F wird es als double interpretiert
// float falsch = 3.14;                    // Warnung/Fehler
float richtig = 3.14F;                     // Korrekt

// Berechnungen
float summe = 10.5F + 20.3F;
```

Worauf achten:

- `F` oder `f` am Ende anhängen: `3.14F`
- Weniger genau als `double`
- In der Praxis wird meist `double` bevorzugt
- Standard-Wert bei Arrays: `0.0F`

String - Zeichenketten

Was ist ein String?

Beschreibung

Ein `String` ist **kein primitiver Datentyp**, sondern eine Klasse (Referenztyp). Er speichert Zeichenketten (Text).

Eigenschaften:

- Unveränderlich (immutable): Einmal erstellt, kann der Inhalt nicht geändert werden
- Wird mit **doppelten** Anführungszeichen erstellt: `TText“`
- Kann beliebig lang sein

Syntax:

```
String variablenName = "Text";
```

Beispiele:

```
// Deklaration und Initialisierung
String name = "Max Mustermann";
String begruessung = "Hallo Welt!";
String leer = "";                           // Leerer String
String mitZahl = "Ich bin 25 Jahre alt";

// String-Konkatenation (Verkettung)
String vorname = "Max";
String nachname = "Mustermann";
String vollername = vorname + " " + nachname;  // "Max Mustermann"

// Mit Zahlen verketten
int alter = 25;
String text = "Ich bin " + alter + " Jahre alt";

// String-Methoden
String wort = "Hallo";
int laenge = wort.length();                 // 5
char erstes = wort.charAt(0);               // 'H'
String gross = wort.toUpperCase();          // "HALLO"
String klein = wort.toLowerCase();          // "hallo"

// Strings vergleichen
```

```java
String a = "Hallo";
String b = "Hallo";
boolean gleich = a.equals(b);              // true
boolean gleichIgnoreCase = a.equalsIgnoreCase("HALLO");  // true

// WICHTIG: Nicht mit == vergleichen!
String s1 = "Test";
String s2 = new String("Test");
boolean falsch = (s1 == s2);               // false (vergleicht Referenzen)
boolean richtig = s1.equals(s2);           // true (vergleicht Inhalt)

// Teilstrings
String text2 = "Hallo Welt";
String teil = text2.substring(0, 5);       // "Hallo"
String ab = text2.substring(6);            // "Welt"

// Suchen in Strings
String satz = "Java ist toll";
boolean enthaelt = satz.contains("ist");   // true
int position = satz.indexOf("ist");        // 5
boolean beginnt = satz.startsWith("Java"); // true
boolean endet = satz.endsWith("toll");     // true

// Ersetzen
String original = "Hallo Welt";
String neu = original.replace("Welt", "Java");  // "Hallo Java"

// Leerzeichen entfernen
String mitLeer = "  Text  ";
String ohneLeer = mitLeer.trim();          // "Text"

// String in Array umwandeln
String liste = "Apfel,Birne,Orange";
String[] fruechte = liste.split(",");      // ["Apfel", "Birne", "Orange"]

// Zahlen zu String
int zahl = 42;
String zahlString = String.valueOf(zahl);  // "42"
// ODER
String zahlString2 = "" + zahl;            // "42"

// String zu Zahl
String nummerText = "123";
int nummer = Integer.parseInt(nummerText); // 123
double komma = Double.parseDouble("3.14"); // 3.14
```

Worauf achten:

- **Doppelte** Anführungszeichen: `TText“`, nicht `'Text'`
- Strings mit `.equals()` vergleichen, **nicht** mit `==`
- `String` mit großem **S** schreiben
- Strings sind unveränderlich: Methoden geben neuen String zurück
- `null` ist möglich, aber (leerer String) ist etwas anderes
- Standard-Wert bei Arrays: `null`

Typkonvertierung

Implizite Konvertierung (Automatisch)

Beschreibung

Java konvertiert automatisch von “kleineremßu “größerem"Datentyp ohne Datenverlust.

Beispiele:

```
// int zu double (automatisch)
int ganzzahl = 5;
double kommazahl = ganzzahl;                // kommazahl = 5.0

// byte zu int (automatisch)
byte klein = 100;
int gross = klein;                          // gross = 100

// char zu int (automatisch)
char buchstabe = 'A';
int code = buchstabe;                       // code = 65 (Unicode-Wert)

// In Berechnungen
int a = 5;
double b = 2.5;
double ergebnis = a + b;                    // ergebnis = 7.5
```

Explizite Konvertierung (Type Cast)

Beschreibung

Von “größeremßu “kleinerem"Datentyp muss explizit gecastet werden. Kann zu Datenverlust führen!

Syntax:

```
zielTyp variable = (zielTyp) quellVariable;
```

Beispiele:

```
// double zu int (Nachkommastellen gehen verloren!)
double kommazahl = 3.7;
int ganzzahl = (int) kommazahl;             // ganzzahl = 3 (nicht 4!)

// long zu int
```

```
long gross = 1000L;
int klein = (int) gross;

// int zu byte (Vorsicht bei gro en Zahlen!)
int zahl = 200;
byte kleinByte = (byte) zahl;              // Overflow!

// Division mit Cast
int a = 5;
int b = 2;
double ergebnis = (double) a / b;          // ergebnis = 2.5

// Runden statt Abschneiden
double wert = 3.7;
int gerundet = (int) Math.round(wert);     // gerundet = 4
```

Worauf achten:

- Cast von double zu int: Nachkommastellen werden **abgeschnitten**, nicht gerundet!
- Bei Überschreitung: Datenverlust oder unerwartete Werte
- Für Rundung: `Math.round()` verwenden
- Implizite Konvertierung: klein → groß
- Explizite Konvertierung (Cast): groß → klein

Übersicht: Primitive Datentypen

Typ	Größe	Wertebereich	Standard
byte	8 Bit	-128 bis 127	0
short	16 Bit	-32.768 bis 32.767	0
int	32 Bit	-2.147.483.648 bis 2.147.483.647	0
long	64 Bit	ca. $-9{,}2 \times 10^{18}$ bis $9{,}2 \times 10^{18}$	0L
float	32 Bit	ca. $\pm 3{,}4 \times 10^{38}$	0.0F
double	64 Bit	ca. $\pm 1{,}7 \times 10^{308}$	0.0
char	16 Bit	0 bis 65.535 (Unicode)	'\u0000'
boolean	1 Bit	true oder false	false

Tabelle 1: Übersicht der primitiven Datentypen in Java

Häufige Fehler

String mit == vergleichen

```
// FALSCH
String a = "Hallo";
String b = "Hallo";
if (a == b) { }  // Vergleicht Referenzen, nicht Inhalt!

// RICHTIG
if (a.equals(b)) { }  // Vergleicht Inhalt
```

Integer-Division

```
// FALSCH - ergibt 2, nicht 2.5
int a = 5;
int b = 2;
double ergebnis = a / b;  // ergebnis = 2.0

// RICHTIG
double ergebnis = (double) a / b;  // ergebnis = 2.5
// ODER
double ergebnis = 5.0 / 2.0;  // ergebnis = 2.5
```

char vs. String verwechseln

```
// FALSCH
char buchstabe = "A";  // Fehler: doppelte Anfuehrungszeichen
String wort = 'Hallo';  // Fehler: einfache Anfuehrungszeichen
```

```

// RICHTIG
char buchstabe = 'A';   // einfache Anfuehrungszeichen
String wort = "Hallo";  // doppelte Anfuehrungszeichen
```

L und F vergessen

```
// FALSCH
long gross = 5000000000;  // Fehler: zu gross fuer int
float zahl = 3.14;        // Warnung: double wird zu float

// RICHTIG
long gross = 5000000000L;  // L am Ende
float zahl = 3.14F;        // F am Ende
```

Overflow nicht beachten

```
// Problem
byte b = 127;
b++;  // b = -128 (Overflow!)

int i = 2147483647;
i++;  // i = -2147483648 (Overflow!)
```

null vs. leerer String

```
String a = null;      // Keine Referenz (keine Methoden aufrufbar!)
String b = "";        // Leerer String (Methoden aufrufbar)

// a.length();        // FEHLER: NullPointerException
int laenge = b.length();  // Funktioniert: laenge = 0
```